

RubricDPO: Preference Tuning a Reranker From Rubric Scores

Conference on Preference Learning, 2025

Abstract

We present RubricDPO, a direct preference optimisation approach to reranker training. Existing systems treat this problem as a single monolithic decision, which wastes computation on inputs that are easy and starves the inputs that are hard. RubricDPO instead allocates effort where the evidence is ambiguous. On RubricPairs we reach 58.8 nDCG@5, against 52.1 nDCG@5 for the PointwiseBCE baseline, while remaining trainable on two 40GB GPUs for ninety minutes.

1 Introduction

Work on reranker training has converged on a small number of architectures that differ more in their training recipes than in their inductive biases. Progress reported on public leaderboards is therefore difficult to attribute, and practitioners choosing a system for a modest deployment have little guidance beyond parameter count. We take the opposite starting point and ask what can be achieved when the compute budget is fixed in advance at two 40GB GPUs for ninety minutes.

Our contribution is RubricDPO, which reorganises reranker training around the observation that most inputs are decided by a small fraction of the available evidence. Rather than scaling the model, we make the decision procedure adaptive. This is a deliberately unfashionable direction: it trades headline capacity for predictable cost, and it is the trade practitioners actually face.

We evaluate on RubricPairs, a collection of 26,700 examples assembled for reranker training. Against the PointwiseBCE baseline, RubricDPO improves nDCG@5 from 52.1 to 58.8. An ablation shows that sampling pairs uniformly instead of by score margin costs 3.9 points of nDCG@5, which locates the gain in the component we claim is responsible rather than in incidental tuning.

The remainder of this paper is organised as follows. Section 2 describes RubricDPO in detail. Section 3 sets out the experimental protocol and the baselines we compare against. Section 4 reports results and ablations. Section 5 discusses the limitations of the approach, of which the most serious is that preference pairs inherit whatever the rubric judge is wrong about, so judge error becomes reranker error with no independent signal to catch it.

Prior work on reranker training divides roughly into two camps. The first scales the encoder and accepts the resulting inference cost, on the assumption that capacity is the binding constraint. The second compresses an expensive model into a cheap one and accepts a fixed quality loss. Neither camp adapts the computation to the individual input, which is the gap RubricDPO occupies.

A second line of work, closer to ours, studies routing between models of different sizes. Those systems typically route on features of the input alone. RubricDPO routes on the disagreement between two scorers that have both already looked at the input, which is a strictly more informative signal and costs one cheap forward pass to obtain.

We make three claims. First, that adaptive allocation recovers most of the quality of an expensive model at a fraction of its cost on RubricPairs. Second, that the confidence estimator can be trained without human labels. Third, that the result is reproducible on two 40GB GPUs for ninety minutes, which we regard as a precondition for the claim being useful rather than merely true.

2 Method

RubricDPO has three stages: a cheap scorer that produces a provisional decision, a confidence estimator that decides whether the provisional decision is safe, and an expensive scorer that is invoked only when it is not. The three stages share an encoder, so the additional parameter cost over the PointwiseBCE baseline is under four percent.

The confidence estimator is the component that makes the direct preference optimisation framing pay. We train it with a margin objective against the disagreement between the cheap and expensive scorers, which requires no additional human annotation: the supervision is manufactured from the two scorers that already exist.

Training proceeds in two phases. The cheap and expensive scorers are trained jointly for the first phase, then frozen while the estimator is fitted. Freezing matters. When all three are trained end to end the estimator learns to mark everything as uncertain, because that minimises loss without ever being penalised for the compute it spends.

Inference is a single forward pass for the majority of inputs. On RubricPairs the expensive scorer fires on roughly one input in five, which is what makes RubricDPO runnable on two 40GB GPUs for ninety minutes at all. We provide the exact thresholds and the schedule used for every reported number in the supplementary material.

The cheap scorer is a two-layer projection over pooled encoder states. We chose pooling over a learned attention head after finding that the two performed within noise of one another on validation, and pooling has no parameters to tune. Where two designs tie, we take the one with fewer knobs, because every knob is a future source of irreproducibility.

The expensive scorer is a full cross-attention pass over the input pair. It is the same architecture as PointwiseBCE, deliberately, so that any difference in the reported $nDCG@5$ is attributable to the routing and not to a better backbone. We initialise both scorers from the same checkpoint for the same reason.

One design detail is worth stating because it cost us several weeks. The confidence estimator must be given the cheap scorer's logits, not its probabilities. With probabilities the estimator saturates on confident inputs and loses the ordering information it needs near the decision boundary, which is precisely the region where routing decisions matter.

3 Experiments

We evaluate on RubricPairs, which contains 26,700 examples. We use the standard split released with the dataset and report $nDCG@5$ on the held-out test portion. No test example is used for threshold selection; thresholds are chosen on a validation split carved out of the training portion.

Our primary baseline is PointwiseBCE, which we retrain under our own protocol rather than quoting published numbers. Retraining moves the PointwiseBCE figure by more than a point relative to the number reported by its authors, which is itself a small argument for retraining baselines rather than

copying tables.

All runs use two 40GB GPUs for ninety minutes. Every reported figure is the mean of five seeds. We report the mean rather than the best seed because the seed-to-seed spread on RubricPairs is roughly one point of nDCG@5 , which is large enough that a best-of-five protocol would manufacture most of a claimed improvement.

We additionally run an ablation in which sampling pairs uniformly instead of by score margin is applied, holding every other setting fixed. This isolates the contribution of the component we claim is doing the work, and it is reported alongside the main result in the next section.

Hyperparameters were selected by random search over sixteen configurations on the validation split, with the search budget fixed in advance and identical for RubricDPO and for PointwiseBCE. Equalising the search budget matters more than the search method: an unequal budget is the most common way a comparison on RubricPairs becomes unfair without anyone intending it.

We also measure wall-clock cost. Every latency figure is taken on two 40GB GPUs for ninety minutes with a warm cache and reported as the median over one thousand queries, since the tail of the distribution is dominated by allocator behaviour rather than by the model. Tail latency is reported separately in the supplementary material.

Statistical testing uses a paired bootstrap over the test queries of RubricPairs with ten thousand resamples. We report a difference as meaningful only when the bootstrap interval excludes zero, which on this dataset requires a gap of roughly one point of nDCG@5 .

4 Results

RubricDPO reaches 58.8 nDCG@5 on RubricPairs. The PointwiseBCE baseline reaches 52.1 nDCG@5 under the identical protocol. The gap is stable across all five seeds and does not depend on the choice of validation split.

The ablation confirms where the gain comes from. With sampling pairs uniformly instead of by score margin, nDCG@5 falls by 3.9 points, recovering most of the distance back to PointwiseBCE. We read this as evidence that the improvement is a property of the direct preference optimisation design rather than of the training schedule.

Cost is the other half of the claim. RubricDPO completes the full RubricPairs test pass on two 40GB GPUs for ninety minutes, which the uncascaded configuration does not. For a team without cluster access this is the difference between a system that can be evaluated and one that can only be cited.

Breaking the RubricPairs test set down by input length shows the gain is not uniform. The longest quartile of inputs accounts for most of the improvement, which is consistent with the direct preference optimisation account: long inputs are where the cheap scorer is least reliable and where routing therefore has the most to decide.

We ran the paired bootstrap described in Section 3 over the difference between RubricDPO and PointwiseBCE. The interval excludes zero comfortably, so we regard the headline gap as real rather than as seed noise. The ablation gap of 3.9 points clears the same bar.

5 Limitations

The most serious limitation of RubricDPO is that preference pairs inherit whatever the rubric judge is wrong about, so judge error becomes reranker error with no independent signal to catch it. We report this prominently because it determines whether the method is appropriate for a given deployment, and it is not visible from the headline nDCG@5 figure.

A second limitation is evaluation scope. RubricPairs is a single dataset in a single language, and we make no claim that the 3.9 point ablation gap would reproduce elsewhere. Cross-dataset transfer is the obvious next study and we have not run it.

Finally, the thresholds that govern when the expensive scorer fires were tuned on RubricPairs and are likely to need retuning on any new corpus. We have not investigated whether they can be set adaptively at inference time.

References

- [1] A. Renner and B. Okafor. Scaling laws for reranker training. *Journal of Retrieval Systems*, 2021.
- [2] C. Lindqvist. PointwiseBCE: a strong baseline for reranker training. In *Proceedings of the Conference on Information Access*, 2020.
- [3] D. Marchetti and E. Yilmaz. The RubricPairs collection. Technical report, 2021.
- [4] F. Novak. On the reproducibility of leaderboard results. *Transactions on Empirical Methods*, 2022.
- [5] G. Adeyemi and H. Sorensen. Evaluation protocols that survive contact with practice. *Annual Review of Applied Machine Learning*, 2023.